NOVA IMS INFORMATION MANAGEMENT SCHOOL

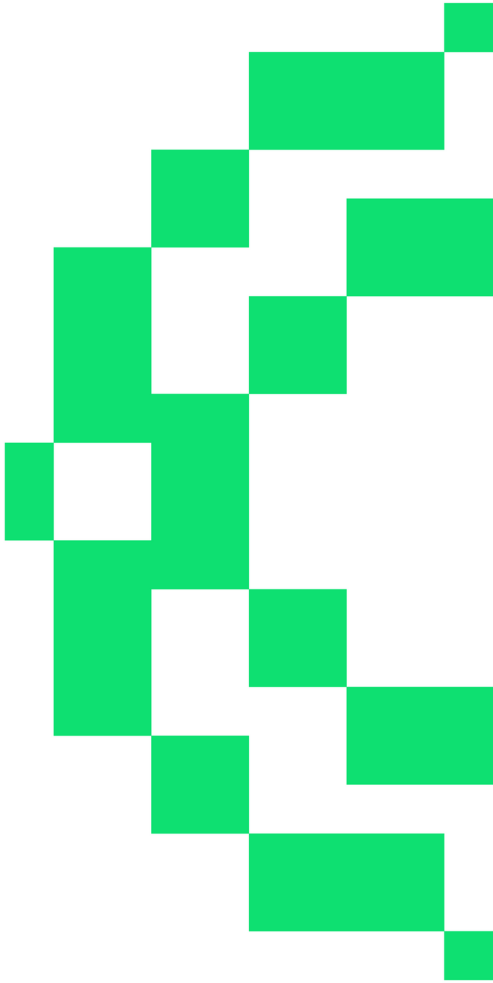
A loop inside a loop

Computation I · Lecture 09

Karina Brotto Rebuli
krebuli@novaims.unl.pt

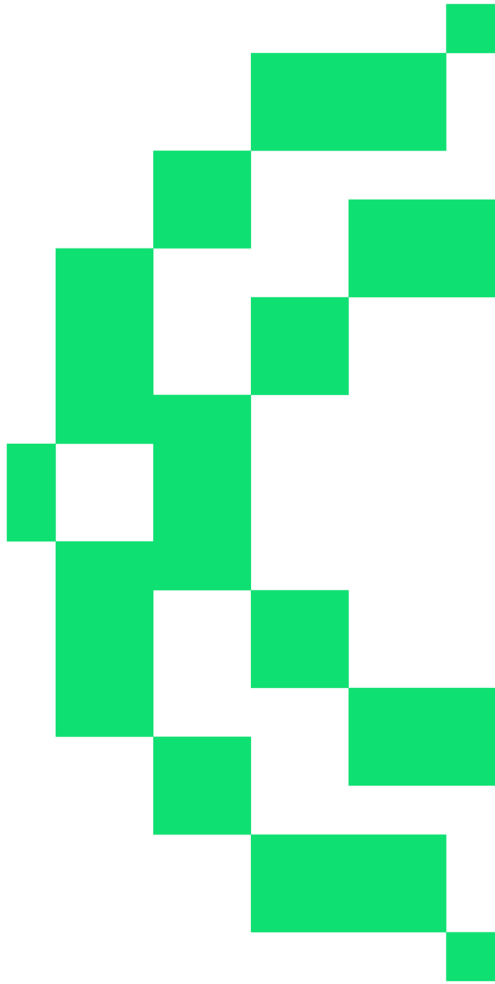
Ten tables, a grid, and a triangle

Last week one loop ran one times table. Today we want all ten tables at once, a whole grid kept in one name, and a triangle whose rows grow. Each needs the same new move: a loop whose body is another loop.



```
"""A loop inside a loop:

the one inside finishes each outer step."""
```



Katherine Johnson

1961 — The same steps, repeated across a grid, and every cell had to be right.

The problem: a spaceflight path was worked out by hand, the same calculation run again and again down a long table of values, row after row.

One slip in a single cell would carry through every value below it — so the work was as much checking each number as producing it.



NASA · Langley Research Center, 1966

Three questions today

Put one loop inside another — what runs, and how often?

Keep a whole grid in one name — and reach any value inside it.

Draw a shape whose rows are not all the same.

01.

A loop inside a loop

One outer step, one whole inner loop

LIVE

The one table that worked

Exercise 9, replayed — the 2 times table

```
for n in range(1, 11):
    print(2 * n)
```

LIVE

Now all ten tables at once

The outer loop picks the table, the inner one runs it

```
for i in range(1, 11):
    for j in range(1, 11):
        print(i * j, end=" ")
    print()
```

```
>>> 1 2 3 4 5 6 7 8 9 10
>>> 2 4 6 8 10 12 14 16 18 20
>>> 3 6 9 12 15 18 21 24 27 30
>>> ...
```

The shape of a nested `for`

The outer loop takes a step and waits — the indentation says who owns whom

```
for i_outer in range(3):
    for i_inner in range(2):
        print(i_outer, i_inner)
    print(i_outer, "finished")
```

THE OUTER HEADER

Takes *one step, then waits* — everything under it runs first.

THE INNER HEADER

A complete loop of its own: *on every single step of the outer loop it runs from start to finish* — the inside finishes first.

THE INDENTATION

How deep a line sits decides which loop owns it: the first print belongs to the inner loop; the second sits one level out — the outer loop runs it once per step, after the inside has finished.

KEEP THIS The inner body runs outer × inner times — and the inside finishes each outer step.

Follow i and j, step by step

Six lines, two counters — watch which one moves

```
for i in range(2):
    for j in range(3):
        print(f"i={i}, j={j}")
```

```
>>> i=0, j=0
>>> i=0, j=1
>>> i=0, j=2
>>> i=1, j=0
>>> i=1, j=1
>>> i=1, j=2
```

LIVE

Predict: how many stars, and how many lines?

On paper before committing

```
for i in range(3):
    for j in range(2):
        print("*", end="")
print()
```

```
for i in range(3):
    for j in range(2):
        print("*", end="")
    print()
```

Diagnose: what does it print?

Every row should be a times table

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i * i, end=" ")
    print()
```

```
>>> 1 1 1
>>> 4 4 4
>>> 9 9 9
```

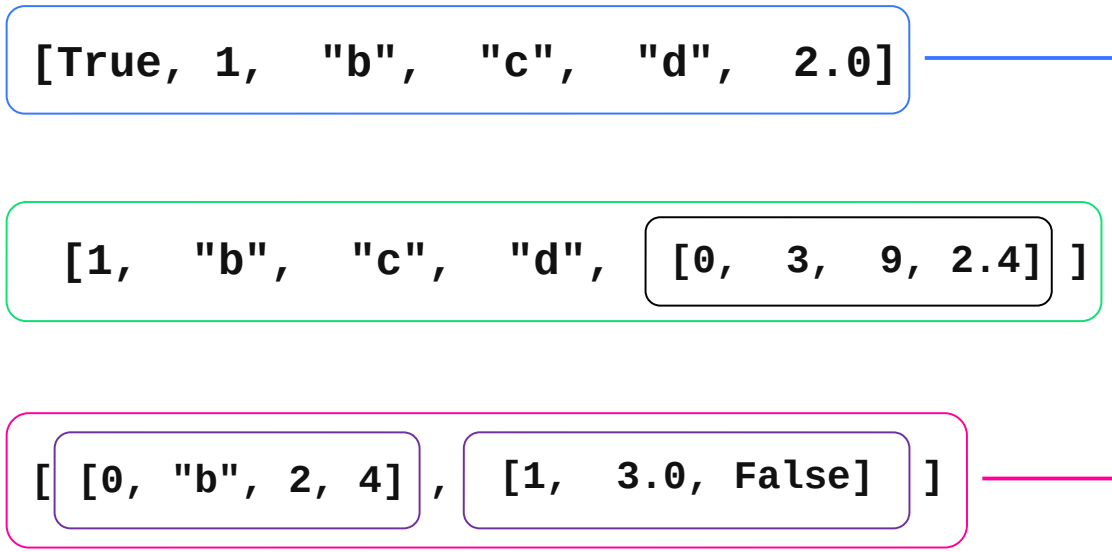
02.

Nested lists and matrices

Lists inside lists — indexed twice, walked by two loops

A list holds values — any values

Mixed types, nested lists, unequal sizes — all legal Python



MIXED IS ALLOWED

A list does not ask its values to match: Boolean, numbers and strings share it happily. Each element of a list has its own data type.

A LIST IS A VALUE TOO

So a list can sit inside another, next to anything else the outer one holds.

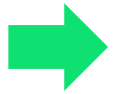
EVEN LISTS OF DIFFERENT SIZES

Nothing forces the inner lists to agree — this one is legal Python.

Reading one value: `g[1][2]`

No new operation — week 7's indexing, applied twice: first to the outer list, second to the inner list

```
g = [[8, "a"],
     [0, 2, 5],
     [True, 9],
     [1]]
```



each element

`g[1][2] = 5`

PICKS THE INNER INDEX — THE PLACE INSIDE IT

A second index, into what came back — g[1][2] is 5. The inner list, then the place in it.

PICKS THE OUTER INDEX — ONE OF THE INNER LISTS

Week 7's indexing, nothing new — g[1] is a whole inner list: [0, 2, 5].

THE OUTER LIST

One name, a list whose items are lists — ragged and mixed, like the previous slide's.

What mathematics calls a matrix

Two dimensions, every row the same length

THE DEFINITION

A matrix is a rectangular arrangement of values in **two dimensions** — rows and columns — in which, at each level, **every element has the same dimension** as its neighbours.

TWO DIMENSIONS

Rows run across, columns run down — one index for each direction, and no third one.

THE SAME DIMENSION AT EVERY LEVEL

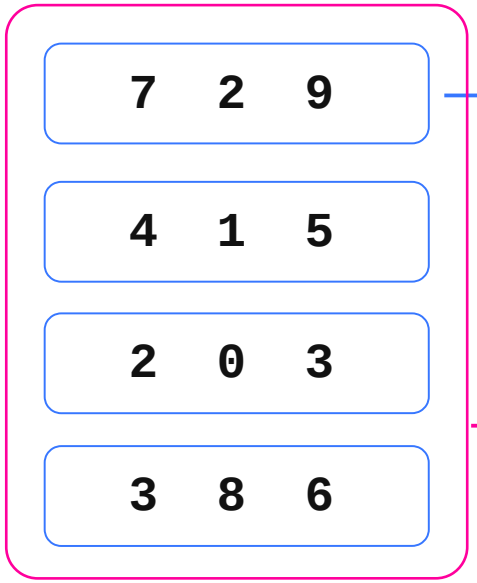
Every row holds the same number of values. The ragged list of the previous slide is legal Python — and it is not a matrix. When the inner lists all have the same size — like `[[1, 3], [2, 4], [5, 6]]` — they gain a name: matrix.

A list whose items are lists of the same length

Stack equal rows and the definition is met — no new syntax

```
row = [7, 2, 9]

m = [ [7, 2, 9],
      [4, 1, 5],
      [2, 0, 3],
      [3, 8, 6] ]
```



A ROW IS A LIST

Week 7's list, exactly: three values in order, one pair of brackets, one name.

AND THAT IS A MATRIX

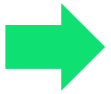
Stack equal rows and the definition is met: two dimensions, every row the same length, one name.
`m` holds four items — and each item is a whole row, a list inside a list.

KEEP THIS A matrix is a list whose items are lists — each matrix item is one row of the grid.
Each row of a matrix has its own elements — each item of each row is an element whose index has two values.

Reading one cell: `m[1][2]`

The same two indexings — on a matrix, the first picks the row, the second the column

```
m = [[7, 2, 9] ,
     [4, 1, 5] ,
     [1, 1, 3] ,
     [6, 5, 2]]
```



each element

`m[1][2] = 5`

`PICKS THE INNER INDEX, IT CAN BE SEEN AS A COLUMN`

A second index, into that row — m[1][2] is 5. The row, then the place in it.

`PICKS THE OUTER INDEX, IT CAN BE SEEN AS A ROW`

Week 7's indexing, nothing new — m[1] is a whole row, the list [4, 1, 5].

`THE WHOLE GRID`

One name, the list of rows — the matrix from the previous slide.

LIVE

Walking the whole matrix, two ways

By element the loops take the rows and their values; by index they count — `i` the rows, `j` the columns.

BY ELEMENT:

```
m = [[7, 2, 9], [4, 1, 5]]
for row in m:
    for value in row:
        print(value, end=" ")
    print()
```

```
>>> 7 2 9
>>> 4 1 5
```

BY INDEX:

```
m = [[7, 2, 9], [4, 1, 5]]
for i in range(len(m)):
    for j in range(len(m[i])):
        print(m[i][j], end=" ")
    print()
```

```
>>> 7 2 9
>>> 4 1 5
```

It runs. Is the total number of inner steps right?

Count every step of the inner loop

```
for i in range(3):
    total = 0
    for j in range(4):
        total = total + 1
print(total)
```

```
>>> 4
```

03.

Loops that draw

Rows that are not all the same

LIVE

The square: every row the same

The outer loop counts the rows, the inner one the columns — 4 × 4 the square, 3 × 6 the rectangle

```
for i in range(4):
    for j in range(4):
        print("*", end="")
    print()
```

```
>>> ****
>>> ****
>>> ****
>>> ****
```

```
for i in range(3):
    for j in range(6):
        print("*", end="")
    print()
```

```
>>> ******
>>> ******
>>> ******
```

LIVE

Now the triangle: rows that grow

The inner loops' lengths depend on the outer counter

```
for i in range(1, 5):
    for j in range(i):
        print("*", end="")
    print()
```

```
>>> *
>>> **
>>> ***
>>> ****
```

```
for i in range(1, 5):
    for j in range(4 - i):
        print(" ", end="")
    for j in range(2 * i - 1):
        print("*", end="")
    print()
```

```
>>>    *
>>>   ***
>>>  *****
>>> *******
```

LIVE

The checkerboard

This time the pattern is in the cell — the if decides each one

```
for i in range(10):
    for j in range(20):
        if (i + j) % 2 == 0:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

```
>>> * * * * * * * * * * 
>>>  * * * * * * * * * * 
>>> * * * * * * * * * * 
>>>  * * * * * * * * * * 
>>> * * * * * * * * * * 
>>>  * * * * * * * * * * 
>>> * * * * * * * * * * 
>>>  * * * * * * * * * * 
>>> * * * * * * * * * * 
>>>  * * * * * * * * * * 
```

What holds when you leave

In one line each

A loop inside a loop

the inner loop finishes before the outer one moves — the inner body runs outer × inner times

A list of lists

`m` holds the rows, and `m[1][2]` reads one value — the row first, then the place inside it

Rows that grow

`range(i)` sets each row's length, and `print()` placement makes the rows — repetition alone cannot

Before the next theoretical lecture

On your own

Think Python — Sections 7.3 Updating variables · 7.4 Looping and counting.

Nested loops and matrices — not in the book. A reference, if you want one: geeksforgeeks.org/python/python-nested-loops

One habit — before you run a loop that draws, write the first two rows by hand. Then compare.

Exercise 1 · A 4-by-4 block of

Write a program that prints a 4-by-4 block of # characters, exactly as the output below shows. Use this lecture's nested for loops.

```
>>> ####
>>> ####
>>> ####
>>> ####
```

Exercise 2 · Each row repeats its own digit

Write a program that prints the four rows below, exactly as the output shows: row 1 shows one 1, row 2 shows two 2s, and so on up to four 4s. Use this lecture's nested for loops.

```
>>> 1
>>> 22
>>> 333
>>> 4444
```

Exercise 3 · The grid on one line

This program should print a 3-by-3 grid of stars. It runs and raises nothing, yet everything comes out on a single line. Find the line to blame, explain when it runs, and write the fix. Everything in it is this lecture's material.

```
for i in range(3):
    for j in range(3):
        print("*", end="")
print()
```

Exercise 4 · Predict what it prints

Read the program below and write the exact lines it prints, in order, and say how many there are. On paper, before any run. Then run it and compare.

```
for i in range(3):
    for j in range(i):
        print(i, j)
```

Exercise 5 · The shrinking triangle

Write a program that prints exactly the triangle below: five stars on the first row, one on the last, each row one star shorter than the one above. Use this lecture's nested for loops.

```
>>> *****
>>> ****
>>> ***
>>> **
>>> *
```

Exercise 6 · The letters of each name

Write a program that prints, for each name in the list, its letters separated by spaces — one name per line, exactly as the output below shows. Start from the list given in the panel, and use this lecture's nested for loops. No range is needed.

```
names = ["Ada", "Grace", "Ken"]
# your loops here
```

```
>>> A d a
>>> G r a c e
>>> K e n
```

Exercise 7 · The even products in each row

Write a program for the 5-by-5 multiplication grid — the products i × j, for i and j from 1 to 5 — that prints, for each row, how many of its products are even, exactly as the output below shows. Use this lecture's nested for loops, with a counter — and decide with care where the counter starts again.

```
>>> 2
>>> 5
>>> 2
>>> 5
>>> 2
```

Exercise 8 · Find the error

This program should print three rows of stars, each ending with its own number — row 0, row 1, row 2. It runs and raises nothing, yet all three end with the same number. Find the line to blame, explain what it does, and write the fix.

```
for i in range(3):
    for i in range(2):
        print("*", end="")
    print(" row", i)
```

Exercise 9 · Predict what it prints

Read the program below and write the exact lines it prints, in order, and say how many there are. On paper, before any run. Then run it and compare.

```
m = [[7, 2, 9], [4, 1, 5]]
print(m[1][0])
print(m[0][2])
print(len(m), len(m[0]))
```

Exercise 10 · Every value on its own line

Write a program that prints every value of the matrix, one value per line, row by row, exactly as the output below shows. Start from the matrix given in the panel and use this lecture's nested for loops. No range is needed.

```
m = [[7, 2, 9], [4, 1, 5]]
# your loops here
```

```
>>> 7
>>> 2
>>> 9
>>> 4
>>> 1
>>> 5
```

Exercise 11 · Check every row

The table was computed by hand: in each row, the last value claims to be the sum of the three before it. One claim is wrong. Write the program that checks every row and prints the verdicts, exactly as the output shows — with this lecture's nested for loops.

```
table = [[7, 2, 9, 18], [4, 1, 5, 11], [3, 8, 6, 17]]
# your loops here
```

```
>>> row 0 ok
>>> row 1 WRONG
>>> row 2 ok
```

Exercise 12 · Print the columns

Write a program that prints the columns of the matrix — the first column on the first line, then the second, then the third, exactly as the output below shows. Use this lecture's m[i][j], and think with care about which counter the outer loop must run.

```
m = [[7, 2, 9], [4, 1, 5], [3, 8, 6]]
# your loops here
```

```
>>> 7 4 3
>>> 2 1 8
>>> 9 5 6
```

Exercise 13 · One more column, one more row

Start from the matrix in the panel. First give the grid one more column: append a 0 to the end of every row. Then give it one more row: append [0, 0, 0, 0]. Print the matrix after each of the two steps, exactly as the output shows — week 7's append does both jobs.

```
m = [[7, 2, 9], [4, 1, 5], [3, 8, 6]]
# your code here
```

```
>>> [[7, 2, 9, 0], [4, 1, 5, 0], [3, 8, 6, 0]]
>>> [[7, 2, 9, 0], [4, 1, 5, 0], [3, 8, 6, 0], [0, 0, 0, 0]]
```

Exercise 14 · The trace

Mathematics calls the sum of the main diagonal the trace of a square matrix. Write the program that computes and prints it, so that it stays right for any square matrix: add a fourth row and column, and it should not change by one character. One loop is enough (tip: the two indexes are the same).

```
m = [[7, 2, 9], [4, 1, 5], [3, 8, 6]]
# your code here
```

```
>>> 14
```

Recap 1 · Predict what it prints

Read the program below and write exactly what it prints — every line, in order. On paper, before any run. Then run it and compare.

```
count = 0
for i in range(1, 4):
    for j in range(i):
        if j % 2 == 0:
            count = count + 1
    print(i, count)
print(count, i, j)
```

Recap 2 · Choose the fragment

Choose the fragment that prints a triangle of stars: one star on the first row, two on the second, three on the third — each row on its own line. One of A to D is right: pick it, and say what each of the others does instead.

```
# A
for i in range(1, 4):
    for j in range(i):
        print("*", end="")
    print()
# B
for i in range(3):
    for j in range(i):
        print("*", end="")
    print()
```

```
# C
for i in range(1, 4):
    for j in range(i):
        print("*", end="")
        print()
# D
for i in range(1, 4):
    for j in range(i):
        print("*", end="")
```

Recap 3 · Debug

The program should print the sum of each row of the matrix — 6, then 15, then 24, one line per row. It runs and raises nothing, yet every line it prints is wrong. Find every line to blame, say what each one does to the sums, and write the fixes.

```
m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
total = 0
for row in range(3):
    for col in range(3):
        total = total + m[col][row]
    print(total)
```

Exercise 15 · Build a list ★

Read the program in the panel, predict the list it prints, then run it and compare. It works — count the lines it takes to build one list from three values.

```
names = ["Ada", "Grace", "Ken"]
lengths = []
for name in names:
    lengths.append(len(name))
print(lengths)
```

Exercise 16 · Build a grid ★

Read the program — today's nested loops, now filling a list. Predict the list it prints, then run it and compare. Count the lines again.

```
pairs = []
for i in range(3):
    for j in range(3):
        pairs.append((i, j))
print(pairs)
```

NOVA IMS INFORMATION MANAGEMENT SCHOOL

```
input("Questions? ")
```

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · Lapcer · IQNet

Cofinanciado por

Portugal 2030 · Lisboa 2030 · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU

© Karina Rebuli, 2026